



Funções Avançadas

Aula 07 · Bloco 2 — Estruturas de Dados

Funções são valores — e isso muda tudo daqui em diante

Nesta aula

1. **A grande ideia:** funções são valores
2. **Arrow functions** — a sintaxe moderna
3. **Callbacks** — passando comportamento
4. Parâmetros com valor padrão
5. **Rest e spread** — os três pontinhos

A grande ideia

Em JavaScript, uma função é **um valor como outro qualquer**.

Pode ser guardada em variável.

Pode ser colocada num array.

E — o que importa de verdade —
pode ser **passada para outra função**.

Função guardada em variável

```
const dobrar = function (n) {  
  return n * 2;  
};  
  
console.log(dobrar(5));      // 10  
console.log(typeof dobrar);  // "function"  
  
const operacoes = [dobrar, function (n) { return n + 1; }];  
console.log(operacoes[0](10)); // 20 – chamando a função do array!
```

Arrow function — a mesma função, três tamanhos

```
const dobrar1 = function (n) {    // 1. function tradicional
  return n * 2;
};

const dobrar2 = (n) => {          // 2. arrow
  return n * 2;
};

const dobrar3 = (n) => n * 2;    // 3. arrow enxuta
```

Corpo de **uma expressão** dispensa `{ }` e `return`.

As regras da forma enxuta

```
const semParametro = () => "olá";           // sem parâmetro: () obrigatório
const umParametro = n => n * 2;              // um: parênteses opcionais
const doisParametros = (a, b) => a + b;      // dois ou mais: () obrigatório

const corpoGrande = (a, b) => {              // mais de uma linha: { } e return
  const soma = a + b;
  return soma / 2;
};
```

✏ **Neste curso:** arrow por padrão em variáveis e callbacks. `function` declarada continua ótima para funções principais com nome.

Callback

Uma função **passada como argumento**
para outra função executar.

É o conceito que destrava o resto do curso:

aula 08 métodos funcionais · **aula 11** eventos · **aula 13**
assincronismo

Callback na prática

```
function processar(lista, acao) { // "acao" será uma função
  for (const item of lista) {
    acao(item); // executa o callback
  }
}

processar(nomes, (nome) => console.log(`Olá, ${nome}!`));
processar(nomes, (nome) => console.log(nome.toUpperCase()));
```

processar define o **esqueleto**. Quem chama define o **que fazer**.

Você já usou callbacks sem saber

```
nums.sort((a, b) => a - b);           // o critério é um callback
notas.forEach((n) => console.log(n));
```

E o clássico do assincronismo — spoiler da **aula 13**:

```
console.log("Início");
setTimeout(() => console.log("...3 segundos depois!"), 3000);
console.log("Fim");
```

⚠ A saída sai **Início**, **Fim** e só então o timeout. Estranhou? É o assunto do Bloco 4.

Parâmetro com valor padrão

```
const saudacao = (nome = "visitante") => `Olá, ${nome}!`;

console.log(saudacao("Maria"));    // Olá, Maria!
console.log(saudacao());           // Olá, visitante!
```

Simples, mas evita um monte de `if (nome === undefined)`.



— o mesmo símbolo, dois papéis

```
// REST – na DEFINIÇÃO: junta vários argumentos num array
const somarTudo = (...numeros) => {
  let total = 0;
  for (const n of numeros) total += n;
  return total;
};
console.log(somarTudo(1, 2, 3, 4, 5));    // 15

// SPREAD – no USO: espalha um array em elementos soltos
console.log(Math.max(...[10, 5, 8]));    // 10
const juntos = [...[1, 2], ...[3, 4], 5]; // [1,2,3,4,5]
```

⚠ Copiar x apelidar

```
const copia = original;      // ✗ segundo NOME para a mesma coisa  
const copia = [...original]; // ✓ cópia de verdade
```

Mudou numa, mudou na "outra" — no primeiro caso.

Vale para objeto também:

```
const copia = { ...original, idade: 21 }
```

Exercícios da aula

Na pasta `aula-07/` :

1. `ex01.js` — quatro funções reescritas como arrow, na forma mais enxuta;
2. `ex02.js` — `aplicar(lista, fn)` — *você acabou de reinventar o `map`* ;
3. `ex03.js` — `filtrar(lista, teste)` — e o *`filter`* ;
4. `ex04.js` — `calculadora(a, b, operacao)` com callback;
5. `ex05.js` — `media(...notas)` com rest, chamada com spread;
6. **Desafio** 🌶️ `ex06.js` — prove a diferença entre `b = a` e `b = [...a]` .

Próxima aula

Aula 08 — Métodos Funcionais

Você reinventou o `map` e o `filter` nos exercícios.

Agora vamos usar os oficiais.